HQC | Experimentos controlados em RTL

Três alterações isoladas, checagens estruturais e testes de referência executados

Origem: arquivo enviado pelo pesquisador `rtl.zip` (SHA-256: e185d58e61c11fdb759f3b299c6dbded8803f0fa39d678d7450c4890fc1014d7). O conteúdo foi extraído para a pasta `original/` sem alterações. Cada variante foi colocada em `mutants/`. Os módulos renomeados permitem comparação lado a lado em testbench.

Status: os testes Python e a inspeção estrutural foram executados. A simulação de eventos do Verilog **não foi executada**, pois Icarus Verilog e Verilator não estavam disponíveis neste ambiente. Foram gerados três testbenches SystemVerilog e um script para execução no Windows. Os resultados a seguir se referem expressamente aos modelos de referência, não à validação HDL final.

Resumo dos experimentos

Caso	Mudança no RTL	Resultado do modelo Python
HT-GF	Alteração na saída registrada da multiplicação GF(2^8)	1 divergência em 65.536 pares; controle matemático independente passou
HT-CTRL	Habilitação de escrita do último byte condicionada ao valor A5	1 escrita suprimida em 256 transações de 46 bytes; done permanece ativo
HT-PEAK	Mudança da prioridade em empate de picos Reed–Muller	31 divergências em 961 pares de magnitudes; exemplo 0x95→0x94

1. HT-GF: integridade de resultado no Reed–Solomon

Arquivo: `hardware/encap/gf_mul.v`. O multiplicador GF(2^8) é utilizado pelo codificador Reed–Solomon (`reed_solomon_encode.v`). O golden mantém o resultado em `out_reg` quando `REG_OUT=1`. A variante altera apenas esse caminho registrado.

```
// Golden (caminho REG_OUT=1)
out_reg <= mul_7;

// Variante (caminho REG_OUT=1)
wire [7:0] mul_selected = ((in_1_reg == 8'hA5) &&
                           (in_2_reg == 8'h5A))
                        ? (mul_7 ^ 8'h01) : mul_7;
out_reg <= mul_selected;
```

Mecanismo: um único bit do resultado é invertido para um par público de estímulos de laboratório. Não há emissão de dados privados e a condição não representa um exploit criptográfico comprovado.

Modelo executado: 65.536 pares de 8 bits; 0 erros do modelo golden frente a um segundo oracle de polinômios; exatamente 1 divergência. Para entradas 0xA5 e 0x5A. o resultado golden é **0xA6** e o alterado é **0xA7**.

Observação sobre a versão: esse módulo realiza redução GF usando 0x11D (x^8+x^4+x^3+x^2+1), enquanto a especificação oficial de 12/10/2026 usa 0x11B (x^8+x^4+x^3+x+1). O oracle usa o polinômio do RTL fornecido; não demonstra conformidade com a revisão 2026 [1].

2. HT-CTRL: escrita final e sinal de conclusão

Arquivo: `hardware/decap/hqc_rmdecod_ctrl.v`. O golden usa `peak_valid_i` diretamente como write-enable. A variante inibe a última escrita se o dado recebido for 0xA5, mantendo inalterada a lógica de done_o.

```
// Golden
assign ram_dout_wr_o = peak_valid_i;

// Variante
assign ram_dout_wr_o = peak_valid_i &
    ~((cnt_out == N1-1) && (peak_dout_i == 8'hA5));
```

Efeito previsto: durante uma transação de 46 bytes (parâmetro N1=46), o endereço 45 deixa de ser escrito quando o dado é A5, embora a conclusão da operação continue sendo sinalizada. Isso viola a propriedade de correspondência entre finalização e integridade da transferência.

Modelo executado: 256 transações, variando o último byte de 0x00 a 0xFF e mantendo os outros 45 fixos; exatamente uma transação contém supressão de escrita. O modelo registra `done_gold=done_mutant=1` no caso afetado.

3. HT-PEAK: fronteira no Reed–Muller

Arquivo: `hardware/decap/hqc_rmdecod_findpeaks.v`. A escolha do pico compara valores absolutos e, em caso de empate, utiliza a maior posição. A variante inverte somente esse desempate.

```
// Golden
assign check_pos = (peak_pos1>peak_pos0);

// Variante
assign check_pos = (peak_pos1<peak_pos0);
```

Exemplo: dois picos positivos de magnitude 7 nas posições 20 e 21. O golden escolhe a posição 21 (`0x95`); a variante escolhe a posição 20 (`0x94`). Para magnitudes 9 e 7, ambos escolhem 20. Assim, o desvio está associado à condição de empate.

Modelo executado: 31×31 pares positivos de magnitude, com posições fixas 20 e 21: 961 casos; 31 diferenças, correspondentes aos empates. O teste em HDL inclui uma sequência de 64 palavras válidas por entrada, para exercitar o submódulo de busca de picos.

4. Evidências e rastreabilidade

Item	Status
Integridade do arquivo recebido	Hash SHA-256 calculado; todos os arquivos em original/ conferidos contra o ZIP
Alterações no source	3 mudanças específicas mais renomeação de módulos para evitar definições duplicadas
Regressão matemática Python	Executada; resultados em model_results.json
Simulação Verilog event-driven	Não executada neste ambiente; testbenches e scripts fornecidos
Teste fim a fim KeyGen/Encaps/Decaps	Não realizado; não há evidência de impacto sobre chave compartilhada
Canal lateral e síntese	Não realizados; não se inferem potência, área ou vazamento físico

5. Reproduzir no Windows com Icarus Verilog

Depois de extrair o ZIP de experimentos, abra o PowerShell na pasta extraída. Com `iverilog` e vvp no PATH:

```
python .\test_reference_models.py
powershell -ExecutionPolicy Bypass -File .\run_iverilog.ps1
```

O script compila separadamente os pares golden/variante e executa os testbenches `tb_gf_mul.sv`, `tb_rm_ctrl.sv` e `tb_findpeaks.sv`. No Linux, utilize `bash ./run_iverilog.sh`. Erros de compilação ou assertions devem ser preservados integralmente no log do artigo.

Após a simulação: confirme a saída esperada do golden e a discrepância específica do mutant; rode também com seeds e cenários adicionais; em seguida, integre a variante ao bloco hierárquico superior e repita os KATs/checagens de rejeição apropriados à versão do HQC. Um sucesso no teste unitário não equivale a validação fim a fim.

6. Critérios para publicação

- Declare versão do RTL e checksum do ZIP. Distingua alteração experimental, trigger, payload e efeito observável.
- Reporte resultados do oracle como **model-level regression** e somente resultados efetivos do Icarus como **RTL simulation**.
- Não atribua recuperação de chave, bypass FO ou vazamento físico a esses três experimentos: nada disso foi demonstrado.
- Inclua métricas de taxa de detecção, falsos positivos, identificação do módulo, cobertura de ativação e custo físico quando houver síntese real.
- Use a especificação de 12/10/2026 apenas como referência para verificar divergências; a arquitetura enviada precisa de atualização ou compatibilidade declarada [1].

Referências para contextualização

- HQC: especificação oficial, revisão de 12/10/2026. Abrir.
- Deshpande et al.: Fast and Efficient Hardware Implementation of HQC, SAC 2023 (repositório). Abrir.

- Goy, Loiseau e Gaborit: A New Key Recovery Side-Channel Attack on HQC with Chosen Ciphertext, 2022. Abrir.
- Goy et al.: Single Trace HQC Shared Key Recovery with SASCA, NIST PQC Conference 2024. Abrir.
- Glabush et al.: Verifiable Decapsulation, CRYPTO 2025. Abrir.
- Schröder, Gast e Guo: Divide and Surrender, USENIX Security 2024. Abrir.
- Simple Power Analysis of Polynomial Multiplication in HQC, ICISSP 2026. Abrir.